

장애를 만들어 정합성을 확인하는 백엔드 개발자

- Kafka 기반 **9개 마이크로서비스**에서 이벤트 순서가 뒤바뀌거나 일부 작업이 실패하는 상황을 재현하며, **Outbox-Saga**를 적용해 데이터 정합성을 맞췄습니다.
- S3 업로드부터 ffmpeg 트랜스코딩, 스트리밍까지 이어지는 **미디어 처리 파이프라인**을 설계하고, 잘못된 파일이나 재처리 상황에도 대응할 수 있도록 만들었습니다.
- 단위·통합·E2E 테스트로 문제를 재현하고 재발을 막았으며, ArchUnit으로 모듈과 계층의 규칙을 **빌드 단계에서 검사**하도록 했습니다.

프로젝트

2026.08

진행중

ESD 플랫폼 — 이벤트 기반 게임 스토어 백엔드

Java 21 · Spring Boot 3.5 · Spring Cloud Gateway · Apache Kafka (KRaft) · MySQL 8 · Redis 7 · Elasticsearch · MongoDB · Docker · Prometheus · Grafana Tempo

Steam, EpicGamesStore, STOVE와 같은 ESD 플랫폼 백엔드입니다. 스튜디오·스토어·다운로드·정산 등 9개의 마이크로서비스를 구성하고 결제 승인, 게임 라이선스 지급, 환불 처리를 구현했습니다.

콘텐츠 등록부터 심의·유통·정산까지 이벤트 기반 아키텍처로 구성했습니다.

■ 라이선스 오환불 방지	라이선스 지급 중 일시적 서버 장애로 재시도 횟수를 소진한 이벤트를 실패로 판단해 8건이 오환불되는 문제가 발생했습니다. 자동환불 조건을 실제 지급 실패로 명확히 분리 하고, 재시도 가능한 이벤트는 보류 후 재처리하도록 변경해 오환불 8건 → 0건 , 정상 결제 건은 1초 이내 전량 지급 되도록 개선했습니다.
■ 이벤트 순서 보장 및 처리량 개선	아웃박스에서 Kafka로 이벤트를 적재하는 과정에서 동일 주문 이벤트의 순서 역전과 직렬 처리에 따른 처리량 저하가 발생해, 적재 배치를 주문번호 단위로 변경해 동일 주문은 순차 처리하고 주문 간 병렬 처리가 가능하도록 개선했습니다. 실패한 이벤트의 재시도 시각을 후행 이벤트에 전파 해 배치가 갈려도 순서가 깨지지 않게 했고, 순서 역전 8건 → 0건 · 처리량 13% 향상 을 이뤘습니다.
■ 이벤트 추적과 폴링 개선	서비스 간 이벤트 처리 과정에서 추적 식별자가 끊겨 전체 흐름을 확인하기 어려운 문제를 해결하기 위해 traceparent를 이벤트와 함께 보존·전달해 6개 서비스를 하나의 트레이스 로 연결했습니다. 이를 통해 발견한 최대 1초의 폴링 대기를 200ms로 줄여 라이선스 표시까지의 p95 1.30/1.06초 → 0.56초 로 개선했습니다.
■ 결제 승인 누락 방지	PG 먹등기를 결제 조회 키로도 사용해 서로 다른 주문의 승인 콜백이 중복으로 오인되는 문제를 해결하기 위해, 주문번호·먹등기·행 잠금 으로 조회·중복 판정·동시성 제어를 분리했습니다. 실제 MySQL 기반 통합 테스트 12건 으로 승인·거절의 정상 및 충돌 경로를 검증했습니다.
■ 장애 지점별 복구	배포·커밋 후 발행·처리 중 장애로 이벤트가 유실되거나 지급 상태가 중간에 멈출 수 있어, 종료 유예 · Outbox 릴레이·재개/만료 스케줄러 로 중단 지점별 복구 경로를 분리했습니다. 재기동·장애 상황을 9종 시나리오 로 나누어 테스트 코드로 작성했습니다.
■ 결제 금액 검증 강화	클라이언트의 결제 금액을 신뢰할 수 없어 카탈로그 재계산·PG 사전등록·승인 금액 대조·중복 방어의 4단계 검증 게이트 를 구성했습니다. 특히 음수 수량으로 기준 금액 자체가 변조될 수 있음을 발견해 상품 수량의 불변식 검증을 추가했습니다.
■ 결함 재현과 테스트 작성	실패를 재현한 뒤 수정하는 방식 으로 개발하며 결함 36건 을 재현 가능한 형태로 기록하고, 수정 후 해당 테스트를 유지했습니다. 단위·통합·인수 테스트를 실행 환경에 따라 분리해 945·241·46건 을 각 층에 맞게 적용했습니다.

2025.09 2025.10 팀 5명 · 4주 뉴스 · 영상 · P2P 담당	JobMate — IT 진로 탐색 및 멘토링 서비스 Java 21 · Spring Boot 3 · Spring Security 6 · Apache Kafka · AWS (S3 · SNS) · Python · ffmpeg · Docker 프로그래머스 백엔드 데브코스의 파이널 팀 프로젝트입니다. 영상 업로드부터 트랜스코딩, 스트리밍 전달까지의 미디어 파이프라인을 담당했습니다.
	업로드 형식 검증 영상 업로드를 구현하면서 S3에 직접 업로드된 객체의 형식이 보장되지 않아 트랜스코딩 단계에서 잘못된 파일을 처리할 수 있는 문제가 있었습니다. 업로드 요청의 파일명으로 허용 확장자를 검증하고 Content-Type을 서명에 포함해, 형식이 맞지 않는 파일은 S3 업로드 단계에서 차단했습니다.
	업로드 중 게시글 작성 트랜스코딩 완료 후 영상 행을 생성해, 업로드 중 게시글을 작성하면 404가 발생하는 문제가 있었습니다. presigned URL 발급 시 READY_TO_UPLOAD 상태의 영상 행을 먼저 생성해 업로드와 게시글 작성을 분리했습니다.
	트랜스코딩 결과 재처리 방지 워커가 트랜스코딩 결과를 같은 버킷에 저장하면서, 저장으로 발생한 S3 이벤트가 다시 워커의 입력으로 전달되는 문제가 있었습니다. transcoded/ 프리픽스로 원본과 결과물을 구분해 워커가 생성한 결과물은 다시 처리하지 않도록 변경했습니다.
	다운로드 경로 추상화 다운로드 API가 클라이언트에 S3 객체 경로를 직접 요구해, 클라이언트가 영상 UUID 외에 서버의 저장 경로까지 알아야 하는 문제가 있었습니다. uuid와 화질만 받도록 변경하고 서버에서 실제 manifest.mpd 경로를 조립해 저장 구조를 클라이언트에서 분리했습니다.
	P2P 시그널링 인증 P2P 연결을 위한 시그널링을 구현하면서, HttpOnly 쿠키의 인증 정보가 WebSocket 통신까지 전달되지 않아 사용자 식별이 되지 않는 문제가 있었습니다. 연결 과정에서 쿠키의 인증 정보를 WebSocket 세션에서 복원해 이후 메시지에서 사용자를 식별하도록 변경했습니다.

2026.06 2026.07	SNG — 수집형 RPG 게임 서버 MSA Kotlin · Spring Boot 3 · Kotlin Multiplatform (JVM/JS/Wasm) · MySQL 8 · Redis 7 · Pekko Cluster Sharding · OCI · Traefik · GitHub Actions Pekko 액터를 바탕으로 구현한 수집형 RPG 게임 서버입니다. 결정론 전투 엔진과 뽑기, 콘텐츠 정산을 구현했습니다.
	이상플레이 탐지를 위한 전투 검증 반복 전투가 핵심인 수집형 모바일게임에서 클라이언트가 전투 결과를 임의로 변경하면 보상과 성장에 영향을 줄 수 있어, 서버가 전투를 재현해 결과를 검증하는 구조를 도입했습니다. 서버와 클라이언트의 계산 차이로 검증 결과가 달라지지 않도록 전투 엔진을 플랫폼 구분 없이 동일한 코드로 구성하고, 여러 실행 환경의 결과가 일치하는지 CI에서 검증하도록 구성했습니다.
	시즌 보상 정산 중단 복구 시즌 종료 후 많은 플레이어에게 보상을 한 번에 처리하는 작업을 하나의 @Transactional로 묶어, 중간에 장애가 발생하면 그동안 처리한 보상까지 모두 롤백되는 문제가 있었습니다. 보상 처리를 200건 단위로 나누어 커밋하고 마지막 처리 위치를 저장하도록 변경해, 1만 건 중 7,000번째에서 장애를 발생시킨 시나리오에서 기존 처리 결과를 유지한 채 재개 하고 전체를 중복 없이 처리 했습니다.
	웹 상점 결제 이벤트 역등 처리 웹 상점과 게임 서비스를 분리하면서 결제 완료를 이벤트로 전달해, 같은 이벤트가 여러 번 전달될 경우 아이템이 중복 지급될 수 있었습니다. 게임 서비스에서 이미 처리한 주문은 다시 지급하지 않도록 역등하게 처리하고, 장애·중복 전달 등 7종 시나리오 와 실제 Redis 환경에서 1회만 지급 됨을 확인했습니다.
	상태 서버의 중단배포 플레이어 상태를 서버 메모리에 유지하는 액터 모델에서 Blue-Green 배포를 적용하면 기존 서버와 새 서버에 같은 플레이어의 액터가 생성되어 상태가 갈라질 수 있어, 상태 정합성을 우선해 단일 컨테이너 중단배포를 선택하고 헬스체크 실패 시 직전 이미지로 복구하도록 구성했습니다.

2026.02
2026.05

Mini-pay — 간편 결제 서비스

Java 17 · Spring Boot 3 · Spring Security 6 · MySQL 8 · k6 · Docker · Next.js

여러 요구사항을 만족하는 페이 서비스를 구현해보며 동시성 제어와 정합성을 이해했습니다.

■ 잔액 동시성 제어	송금 기능을 구현하면서 잔액 조회와 차감이 분리된 경우 동시에 들어온 요청이 같은 잔액을 읽어 갱신할 수 있어, 잔액 정합성이 깨지는 문제가 발생했습니다. 커스텀 레포지토리에서 <code>balance ≥ :amount</code> 조건과 차감을 하나의 UPDATE로 처리해 <u>잔액 검증과 차감을 원자적으로 수행</u> 하도록 변경했습니다.
■ 배치 입금 합산	이체의 출금과 입금을 비동기로 분리하면서 동일 수신자에게 여러 건이 몰리면 입금 건마다 계좌를 갱신해 <u>DB I/O가 N건 발생</u> 하는 문제가 생겼습니다. 수신자별 입금액을 먼저 합산해 <u>N건 → 1건</u> 으로 잔액을 갱신하도록 변경했습니다.
■ 이체 실패 시 자동 보상	출금과 입금을 비동기로 분리한 뒤 수신 계좌 조회가 반복해서 실패하면 송금인의 돈만 빠진 채 이체가 종료될 수 있어, 실패한 입금을 재시도해 <u>5회 실패 후 자동 보상</u> 하도록 처리 상태와 환불 경로를 구성했습니다. 대상 계좌 삭제 상황에서도 재시도와 보상까지 처리되도록 해 출금 후 입금에 실패한 금액이 남지 않도록 했습니다.
■ 출금 원장 정합성	출금은 먼저 완료되지만 수신인 원장은 입금 성공 시점에 함께 생성되는 구조여서, 입금 실패나 보상 상황에서는 잔액과 거래내역이 달라지는 문제가 발생했습니다. 출금 시점에 송금인 원장을 기록하고 배치에서는 수신인 원장만 생성하도록 변경해 <u>잔액과 거래내역의 정합성</u> 을 맞췄습니다.

교육 및 경험

2026.07 진행중	코디세이 AI 올인원 2기 CS와 AI 기반 지식을 동료 학습을 통해 다지고 있습니다.
2025.05 2025.10	백엔드 데브코스 — 프로그래머스 네 번의 팀 프로젝트를 거치며 팀장을 보조해 코드 리뷰와 스터디를 이끌었습니다.
2024.09 2024.12	글로벌 인턴십 — TAFE NSW 비즈니스 영어를 집중 학습하고, 노르웨이-호주 상공회의소 임원의 요구사항을 영어로 직접 조율해 수출입 지원서 관리 서비스를 기간 내 인도했습니다. 운영 중이던 PHP 시스템에 기능을 끼워 넣는 대신 Spring Boot로 별도 서비스를 만드는 방안을 제안해 승인받았습니다.
2024.01 2024.02	단기 해외 연수 — Box Hill Institute 네트워크와 리눅스 기초를 배우고 Terraform으로 네트워크 실습을 했습니다.
2020.03 2025.03	영진전문대학교 — 컴퓨터 정보계열, 전문학사 여러 프레임워크로 웹 프로젝트를 수행하며 부족한 부분을 주도적으로 메웠습니다.
2018.05 2018.11	스마트 웹 & 콘텐츠 개발자 양성과정 — 예담 직업전문학교 프로그래밍과 웹의 기초 지식을 익혔습니다.